

Backend Services & Routes Implementation Summary

Overview

This document summarizes the backend services and routes implementation for the Chicken Processing & Packaging Sales Platform. The backend is built with Express.js, TypeScript, and Firebase/Firestore.

Architecture

Technology Stack

- **Runtime:** Node.js
- **Framework:** Express.js
- **Language:** TypeScript
- **Database:** Firebase Firestore
- **Authentication:** Firebase Auth
- **Security:** Helmet, CORS

Project Structure

```
chicken-processing-backend/  
├── src/  
│   ├── app.ts           # Main Express application setup  
│   ├── server.ts        # Server entry point  
│   ├── config/  
│   │   └── firebase.ts  # Firebase configuration  
│   ├── controllers/     # Request handlers  
│   ├── domain/          # Domain models  
│   ├── dto/             # Data Transfer Objects  
│   ├── middleware/  
│   │   ├── auth.ts      # Authentication middleware  
│   │   ├── errorHandler.ts # Error handling middleware  
│   │   └── validation.ts # Request validation  
│   ├── models/          # Firestore data models  
│   ├── routes/          # API route definitions  
│   ├── services/        # Business logic services  
│   └── types/           # TypeScript type definitions
```

Implemented Routes

1. Authentication Routes (</api/auth>)

File: <src/routes/auth.ts>

- User registration and login

- Password reset
- Token refresh
- User profile management

2. User Management Routes (/api/users)

File: `src/routes/users.ts`

- CRUD operations for users
- User search and filtering
- Role-based access control
- User statistics

3. Product Routes (/api/products)

File: `src/routes/products.ts`

- Product catalog management
- Product search and filtering
- Category management
- Price management
- Product images

4. Order Routes (/api/orders)

File: `src/routes/orders.ts`

- Order creation and management
- Order status tracking
- Order history
- Order cancellation
- Order refunds

5. Inventory Routes (/api/inventory)

File: `src/routes/inventory.ts`

- Stock management
- Inventory tracking
- Batch management
- Expiration tracking
- Inventory reports
- Low stock alerts

6. Production Routes (/api/production)

File: `src/routes/production.ts`

- Production cycle management
- Production logging

- Daily production reports
- Production analytics

7. Medication Routes (/api/medication)

File: `src/routes/medication.ts`

- Medication tracking
- Dosage management
- Withdrawal period tracking
- Medication reports

8. Harvest Routes (/api/harvest)

File: `src/routes/harvest.ts`

- Harvest scheduling
- Processing records
- Batch tracking
- Quality metrics
- Yield calculations

9. Payment Routes (/api/payments)

File: `src/routes/payments.ts`

- Payment processing
- Payment history
- Refund management
- Payment methods

10. Quality Control Routes (/api/quality)

File: `src/routes/quality.ts`

- Quality inspections
- Defect tracking
- Quality metrics
- Compliance checks
- Quality reports

11. Configuration Routes (/api/configuration)

File: `src/routes/configuration.ts`

- System configuration management
- Feature flags
- Business rules
- Integration settings

12. System Logs Routes (/api/system-logs)

File: `src/routes/systemLogs.ts`

- System activity logs
- Audit trails
- Error logs
- Performance logs

13. API Keys Routes (/api/api-keys)

File: `src/routes/apiKeys.ts`

- API key management
- Key generation
- Key revocation
- Usage tracking

14. Notification Config Routes (/api/notification-configs)

File: `src/routes/notificationConfigs.ts`

- Notification settings
- Channel configuration
- Template management

15. Employee Routes (/api/employees)

File: `src/routes/employees.ts`

- Employee profile management
- Schedule management
- Shift requests
- Employee statistics
- Department management

16. CRM Routes (/api/crm)

File: `src/routes/crm.ts`

- Customer profile management
- Customer segmentation
- Loyalty program management
- Feedback and complaints
- Promotional campaigns
- Customer analytics

17. Notifications Routes (/api/notifications)

File: `src/routes/notifications.ts` (NEW)

- Get user notifications
- Get unread notification count
- Mark notifications as read
- Notification preferences

18. Analytics Routes (</api/analytics>)

File: [src/routes/analytics.ts](#) (NEW)

- Dashboard summary
- Sales analytics
- Financial analytics
- Profit and loss statements
- Sales trends
- Average order value (AOV) and customer acquisition cost (CAC)
- Sales forecasting
- Analytics reports

19. Data Management Routes (</api/data-management>)

File: [src/routes/dataManagement.ts](#) (NEW)

- Backup configurations
- Export configurations
- Validation rules
- Integrity constraints
- Migration plans
- Archive policies
- Operation logs
- System statistics

Implemented Services

1. Authentication Services

- [authService.ts](#) - Basic authentication
- [authServiceV2.ts](#) - Enhanced authentication with validation

2. User Services

- [UserService.ts](#) - User management business logic

3. Product Services

- [productService.ts](#) - Product management

4. Order Services

- [orderService.ts](#) - Order processing

5. Inventory Services

- `inventoryService.ts` - Inventory management

6. Production Services

- `productionService.ts` - Production management

7. Medication Services

- `medicationService.ts` - Medication tracking

8. Harvest Services

- `harvestProcessingService.ts` - Harvest and processing

9. Payment Services

- `paymentService.ts` - Payment processing

10. Quality Control Services

- `qualityControlService.ts` - Quality management

11. Configuration Services

- `configurationService.ts` - System configuration

12. System Log Services

- `systemLogService.ts` - Logging service

13. API Key Services

- `apiKeyService.ts` - API key management

14. Notification Services

- `notificationConfigService.ts` - Notification configuration
- `notificationService.ts` - Notification sending and management

15. Employee Services

- `employeeService.ts` - Employee management

16. CRM Services

- `CRMService.ts` - Customer relationship management

17. Analytics Services

- `analyticsService.ts` - Analytics and reporting

18. Data Management Services

- `dataManagement/dataManagementService.ts` - Data management operations

Data Models

Core Models

- `User.ts` - User accounts
- `Product.ts` - Product catalog
- `Order.ts` - Orders
- `Inventory.ts` - Inventory items
- `Production.ts` - Production cycles
- `Employee.ts` - Employees

Supporting Models

- `CustomerProfile.ts` - Customer profiles
- `Medication.ts` - Medications
- `HarvestProcessing.ts` - Harvest records
- `QualityControl.ts` - Quality inspections
- `Notification.ts` - Notifications
- `SystemLog.ts` - System logs
- `ApiKey.ts` - API keys
- `Role.ts` - User roles
- `SystemConfiguration.ts` - System settings
- `NotificationConfig.ts` - Notification settings

Data Management Models

- `BackupConfig.ts` - Backup configurations
- `ExportConfig.ts` - Export configurations
- `ValidationRule.ts` - Validation rules
- `IntegrityConstraint.ts` - Data integrity constraints
- `MigrationPlan.ts` - Data migration plans
- `ArchivePolicy.ts` - Data archive policies
- `DataOperationLog.ts` - Operation logs

CRM Models

- `LoyaltyProgram.ts` - Loyalty programs
- `FeedbackComplaint.ts` - Customer feedback
- `PromotionalCampaign.ts` - Marketing campaigns

Analytics Models

- `Analytics.ts` - Analytics data structures

Middleware

Authentication Middleware (**auth.ts**)

- Token verification
- User authentication
- Role-based authorization
- Admin access control

Error Handler Middleware (**errorHandler.ts**)

- Centralized error handling
- Error logging
- Client-friendly error responses
- 404 handling

Validation Middleware (**validation.ts**)

- Request body validation
- Schema validation
- Input sanitization

Security Features

1. **Helmet**: Security headers for HTTP responses
2. **CORS**: Cross-origin resource sharing configuration
3. **Firestore Auth**: Secure authentication
4. **Role-based Access**: Admin and user role separation
5. **Input Validation**: Request data validation
6. **Error Handling**: Secure error responses

API Endpoints Summary

Route Prefix	Description	Auth Required	Admin Only
/api/auth	Authentication	Varies	No
/api/users	User management	Yes	Partial
/api/products	Product catalog	Yes	Partial
/api/orders	Order management	Yes	No
/api/inventory	Inventory management	Yes	Partial
/api/production	Production tracking	Yes	Partial
/api/medication	Medication tracking	Yes	Partial
/api/harvest	Harvest processing	Yes	Partial
/api/payments	Payment processing	Yes	No
/api/quality	Quality control	Yes	Partial

Route Prefix	Description	Auth Required	Admin Only
/api/configuration	System configuration	Yes	Yes
/api/system-logs	System logs	Yes	Yes
/api/api-keys	API key management	Yes	Yes
/api/notification-configs	Notification settings	Yes	Yes
/api/employees	Employee management	Yes	Partial
/api/crm	Customer relationship	Yes	Partial
/api/notifications	User notifications	Yes	No
/api/analytics	Analytics & reports	Yes	Yes
/api/data-management	Data management	Yes	Yes

Recent Additions

1. Notifications Route (/api/notifications)

Purpose: Manage user notifications

Features:

- Get user notifications with pagination
- Get unread notification count
- Mark notifications as read

Endpoints:

- GET / - Get user notifications
- GET /unread-count - Get unread count
- PUT /:id/read - Mark as read

2. Analytics Route (/api/analytics)

Purpose: Business analytics and reporting

Features:

- Dashboard summary
- Sales analytics with date ranges
- Financial analytics and P&L statements
- Sales trends and forecasting
- Customer acquisition metrics

Endpoints:

- GET /dashboard - Dashboard summary
- GET /sales - Sales analytics
- GET /financial - Financial analytics

- GET /profit-loss - P&L statement
- GET /sales-trends - Sales trends
- GET /aov-cac - AOV and CAC metrics
- GET /forecast - Sales forecasting
- POST /reports - Save analytics report
- GET /reports/:id - Get analytics report

3. Data Management Route (/api/data-management)

Purpose: Data management operations

Features:

- Backup configuration management
- Export configuration management
- Validation rule management
- Integrity constraint management
- Migration plan management
- Archive policy management
- Operation logs
- System statistics

Endpoints:

- GET /backup-configs - List backup configs
- GET /backup-configs/:id - Get backup config
- POST /backup-configs - Create backup config
- PUT /backup-configs/:id - Update backup config
- GET /export-configs - List export configs
- GET /export-configs/:id - Get export config
- POST /export-configs - Create export config
- GET /validation-rules - List validation rules
- GET /validation-rules/:id - Get validation rule
- POST /validation-rules - Create validation rule
- GET /integrity-constraints - List constraints
- GET /integrity-constraints/:id - Get constraint
- POST /integrity-constraints - Create constraint
- GET /migration-plans - List migration plans
- GET /migration-plans/:id - Get migration plan
- POST /migration-plans - Create migration plan
- GET /archive-policies - List archive policies
- GET /archive-policies/:id - Get archive policy
- POST /archive-policies - Create archive policy
- GET /operation-logs - Get operation logs
- GET /statistics - Get system statistics

Testing

Test files are available for various services:

- `test-analytics.js`
- `test-crm.js`
- `test-configuration.js`
- `test-data-management.js`
- `test-employees.js`
- `test-harvest.js`
- `test-inventory.js`
- `test-medication.js`
- `test-order-system.js`
- `test-payment.js`
- `test-quality.js`

Environment Configuration

Required environment variables (`.env`):

```
PORT=5000
NODE_ENV=development
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_PRIVATE_KEY=your-private-key
FIREBASE_CLIENT_EMAIL=your-client-email
CORS_ORIGIN=http://localhost:3000,http://localhost:5173
```

Getting Started

1. Install dependencies:

```
npm install
```

2. Set up environment variables in `.env`

3. Run in development mode:

```
npm run dev
```

4. Build for production:

```
npm run build
```

5. Run in production:

```
npm start
```

API Documentation

The API follows RESTful conventions:

- **GET** - Retrieve resources
- **POST** - Create resources
- **PUT** - Update resources
- **DELETE** - Remove resources

All responses follow this format:

```
{
  "success": true,
  "data": {},
  "message": "Optional message"
}
```

Error responses:

```
{
  "success": false,
  "message": "Error description",
  "error": "Detailed error (development only)"
}
```

Future Enhancements

1. **API Documentation:** Add Swagger/OpenAPI documentation
2. **Rate Limiting:** Implement API rate limiting
3. **Caching:** Add Redis caching for frequently accessed data
4. **WebSocket:** Real-time notifications via WebSocket
5. **File Upload:** Enhanced file upload handling
6. **Search:** Full-text search capabilities
7. **Batch Operations:** Bulk operations for efficiency
8. **Webhooks:** External webhook support
9. **GraphQL:** Optional GraphQL API layer
10. **Microservices:** Potential microservices architecture

Notes

- All routes require authentication unless specified

- Admin-only routes are protected by `authorizeAdmin` middleware
- Firebase Firestore is used as the primary database
- All timestamps are stored in UTC
- Pagination is supported on list endpoints
- Search and filtering available on most list endpoints